

DETECTING SPARSITY PATTERNS IN TENSOR DATA

PETER A. BROOKSBANK, MARTIN D. KASSABOV, AND JAMES B. WILSON

ABSTRACT. This article introduces a class of efficiently computable *sparsity patterns* for tensor data. The class includes familiar patterns such as block-diagonal decompositions explored in statistics and signal processing [1, 9–12], low-rank tensor decompositions [13], and Tucker decompositions [23]. It also includes a new family of sparsity patterns—not known to be detectable by current methods—that can be thought of as continuous decompositions approximating curves and surfaces. We present a general algorithm to detect sparsity patterns in each class using a parameter we call a *chisel* that tunes the search to patterns of a prescribed shape. We also show that the patterns output by the algorithm are essentially unique.

1. INTRODUCTION

This paper concerns the detection of *sparsity patterns* in tensor data. A *tensor* is any data with an interpretation $\gamma : U_{-m}^* \times \cdots \times U_{-1}^* \times U_1 \times \cdots \times U_\ell \rightarrow U_0$ as a multilinear mapping of vector spaces $U_0, \dots, U_\ell$ and dual spaces $U_{-1}^*, \dots, U_{-m}^*$, collectively called *modes*, into a space U_0 called the *base*, cf. [21, Definition 3.3].

A *sparsity pattern* for γ exhibited by decompositions

$$U_a = X_{a1} \oplus X_{a2} \oplus \cdots \oplus X_{ak_a}, \quad -m \leq a \leq \ell,$$

is a set $\Delta \subset \prod_a [k_a]$ satisfying

$$(1) \quad (i_{-m}, \dots, i_\ell) \notin \Delta \implies \gamma(X_{-mi_{-m}}^*, \dots, X_{-1i_{-1}}^*, X_{1i_1}, \dots, X_{\ell i_\ell}) \subseteq X_{0i_0}.$$

By fixing a basis $\{e_{a,i_a} \mid 1 \leq i_a \leq d_a\}$ for each U_a one can represent γ as a multiway array Γ with the following entries:

$$(2) \quad \Gamma_{i_1 \dots i_\ell}^{i_{-m} \dots i_{-1} i_0} := e_{0i_0}^\top \gamma(e_{-mi_{-m}}^\top, \dots, e_{-1i_{-1}}^\top, e_{1i_1}, \dots, e_{\ell i_\ell}).$$

Arrays are a common tensor data structure and a useful visual tool, but it is important to remember the underlying multilinear interpretation. Tensors on three modes can be visualized as point clouds in 3-dimensional space, as in Figure 1.

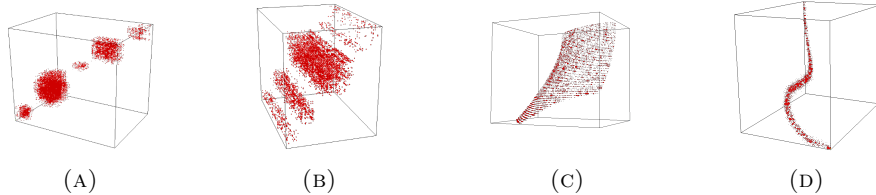


FIGURE 1. Point clouds of tensor arrays recovered by our algorithms that exhibit various types of sparsity patterns.

Date: January 15, 2026.

2020 Mathematics Subject Classification. 20D45, 20D15, 08C05, 68Q65.

Each of the plots reveals a sparsity pattern exhibited by some decompositions of the three modes. For instance, the point cloud in Figure 1(a) illustrates the sparsity pattern $\Delta = \{(i, i, i) \mid 1 \leq i \leq 5\}$ exhibited by decompositions of each mode into five direct summands. The surface and curve in Figure 1(c) and Figure 1(d), respectively, show sparsity patterns exhibited by more granular decompositions.

It is important to emphasize that tensors are generally not recorded in reference frames that naturally exhibit sparsity patterns, so the critical task is to find suitable basis changes for the modes and base of the tensor. The images in Figure 1 were generated by our algorithms, which found such basis changes in synthetically created and scrambled tensor data.

Block decompositions. Many problems in tensor research reduce to finding decompositions of the mode spaces that exhibit block structures of one form or another. For a modern survey of tensor decompositions, see [20]. For instance, block diagonal structures of the sort depicted in Figure 1(A) are sought in blind source separation [12], where higher-order statistics such as moments or cumulants of independent events occur as distinct clusters on the diagonal. Figure 1(B) exemplifies another common type of decomposition that focuses on a pair of modes and seeks decompositions along their common face. There are many applications of “face block” decompositions, including polynomial and algebra factorization [15, 25–27], community detection [1, 2], simultaneous block diagonalization of matrices [9–11, 22, 26], and low-rank approximation problems [13].

Continuous decompositions. Our algorithms work for a broader class of computable sparsity patterns that also includes the point clouds depicted in Figure 1(C) and (D) in which the data clusters around a surface, or intersection of surfaces. One can think of the corresponding tensor decompositions as continuous analogues of the more traditional block decompositions.

Features of our approach. Our approach is inspired by distributive products in the theory of (nonassociative) rings, where Lie theory plays an essential role. In a sense, our results are the outcomes of a study of “regular semisimple elements of the Lie algebra of derivations of a tensor” [17, Chapter IV]. The result is a single algorithm that can be tuned by parameters to control the shape of the sparsity pattern sought, and also to account for additional symmetries that may be present. Indeed, our approach unifies the outcomes of various existing tensor methods, such as Tucker decompositions [23], and some cases of Higher-Order Singular Value Decompositions [13, 14]. Details of how our methods can recover these decompositions are provided in Section 7.5, but we stress that in no way do we propose to compete with these specialized and highly optimized tensor techniques.

Software. We have implemented our algorithms in the Julia programming language [3]. Source code and documentation are available on GitHub [8], including interactive notebooks that illustrate the main features of our methods. These notebooks feature variations on the experiments with synthetic data discussed throughout the paper, as well as examples with real data.

Adapting to noise. Our algorithms can uncover approximations to sparsity patterns, function reasonably well under a range of noisy conditions, and tolerate a number of outliers to a sparsity pattern. Recently, Nick Vannieuwenhoven has studied stability of variations of our algorithms under noise [24].

Outline of paper. In Section 2 we discuss popular data structures for tensors called multiarrays and compare two natural tensor interpretations that use them. Section 3 defines sparsity patterns and explains how they can be visualized in a multiarray interpretation using tensor contractions.

In Section 4, we introduce chisels and define the class of sparsity patterns our methods are designed to detect. Section 5 introduces *derivations* as a tool to connect chisels and sparsity patterns to the underlying mode decompositions. In particular, we prove the result that ensures our methods work (Theorem 5.5).

In Section 6, we present our ‘chiseling’ algorithms: given a tensor and a chisel, we determine, constructively, whether there is a decomposition of the modes of the tensor that exhibits a sparsity pattern corresponding to the chisel. Section 7 takes a closer look at certain chisels we have found useful in exploring tensors, and explains why they reveal the patterns for which they are designed. It also elucidates the connection between our methods and established tensor decomposition techniques.

The remainder of the paper contains an abridged theory of chisels and derivations needed to explain additional features of our algorithms. Section 8 situates our approach within a general spectral theory of tensors [16] so that, among other things, our methods can be adapted to account for additional symmetries that may be present. Section 9 discusses the important issues of uniqueness and reproducibility. We show that algebraic properties of derivations of a ‘universal’ chisel ensure that outputs of our algorithm over the complex field are unique.

2. MULTIARRAYS

We saw in the introduction that array representations of tensors are very useful for visualization, and we shall continue to use them throughout. Thus, when we say “ Γ is a tensor,” we will usually be thinking that Γ is an abstract array (in the sense of the Julia language, for instance), and we shall work with Γ as a traditional multiarray. However, we also noted, when using arrays, that it is important not to lose sight of the underlying multilinear interpretation.

We emphasize that point by considering two common tensor interpretations that use multiarrays. Given spaces $U_a = \mathbb{K}^{d_a}$ for $a \in [\ell] := \{1, \dots, \ell\}$, where each $u_a \in U_a$ is a function $[d_a] \rightarrow \mathbb{K}$ sending $i \mapsto u_{ai}$, the tensor interpretation

$$(3) \quad u_1 \otimes \cdots \otimes u_\ell \quad := \quad (i_1, \dots, i_\ell) \mapsto u_{1i_1} \cdots u_{\ell i_\ell}.$$

outputs multiarrays in $U_0 = \mathbb{K}^{d_1 \times \cdots \times d_\ell}$. Given a multiarray Γ , the interpretation

$$(4) \quad \langle \Gamma \mid u_1, \dots, u_\ell \rangle \quad := \quad \sum_{i_1 \in [d_1]} \cdots \sum_{i_\ell \in [d_\ell]} \Gamma_{i_1 \dots i_\ell} u_{1i_1} \cdots u_{\ell i_\ell}$$

outputs scalars in $U_0 = \mathbb{K}$.

In the case $\ell = 2$, $u_1 \otimes u_2$ is a $(d_1 \times d_2)$ -matrix. It is more common to interpret equation (4) as $\langle u_1 \mid \Gamma \mid u_2 \rangle = \gamma(u_1, u_2) = \langle \Gamma \mid u_1^\top, u_2 \rangle$ for a $(d_1 \times d_2)$ -matrix Γ .

Figure 2 illustrates both forms of tensor interpretation for 3-way arrays. Vectors are drawn as colored ribbons, matrices are rectangles, and 3-way arrays are boxes.

2.1. Partial evaluations of tensors. In the same way that a matrix product can be computed as a sequence of binary products, so multilinear maps can be computed in a sequence of partial evaluations. Let $A = \{a_1 < \cdots < a_\ell\} \subset [\ell]$ be a

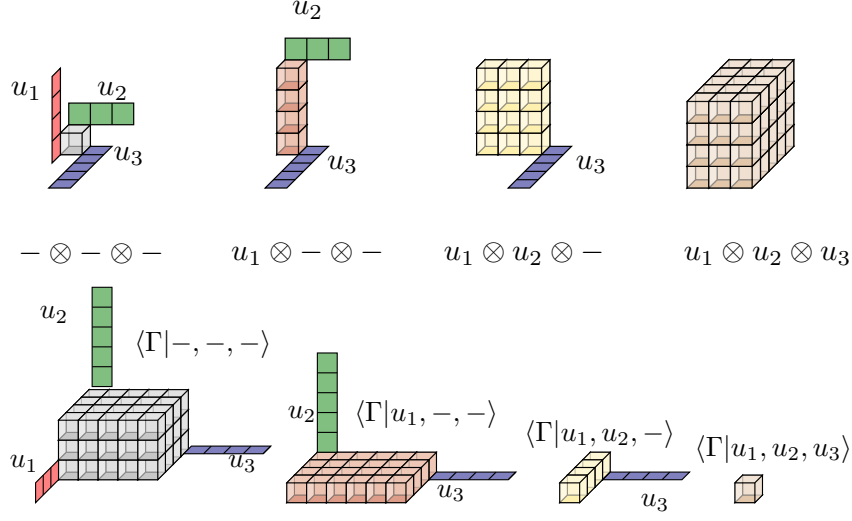


FIGURE 2. Tensor interpretations visualized as sequences of partial evaluations. On the top is the interpretation in equation (3) and on the bottom is the interpretation in equation (4).

set of modes, and let $u_a \in U_a$ for $a \in A$. A *partial evaluation* at $u_A = (u_a : a \in A)$, sometimes called a *contraction*, is defined as follows:

$$(5) \quad \langle \Gamma \mid u_A \rangle := \langle \Gamma \mid -, u_{a_1}, - \cdots -, u_{a_k}, - \rangle.$$

Observe that partial evaluation produces a new tensor interpretation with modes $\{U_a : a \notin A\}$. Figure 2 illustrates sequences of partial evaluations for the multiarray interpretations in equations (3) and (4). Observe that partial evaluations reduce the valence of the tensor but not necessarily the size of the data structure.

2.2. Notational convention. For both exposition and visualization we shall henceforth work just with tensor interpretations like the one in equation (4) with no dual spaces and a 1-dimensional base. When all vector spaces are finite-dimensional, one can always pass to an equivalent interpretation of this form. Specifically, if we have any multilinear function $\gamma : U_{-m}^* \times \cdots \times U_{-1}^* \times U_1 \times \cdots \times U_\ell \rightarrow U_0$, we can use the interpretation $\langle \Gamma \mid u_{-m}^\top, \dots, u_{-1}^\top, u_0^\top, u_1, \dots, u_\ell \rangle$, where $u_0 = \gamma(u_{-m}, \dots, u_{-1}, u_1, \dots, u_\ell)$.

2.3. Changing bases. The original mode bases relative to which a tensor is recorded is considered arbitrary. In particular, even if a multilinear map γ exhibits a sparsity pattern for some decompositions of its modes, there is no reason to suppose that our chosen multiarray representation Γ of γ is recorded relative to bases that respect these decompositions. Indeed the main task, for a given Γ , is to find bases of the modes that do respect the decompositions of some sparsity pattern for Γ , and then to rewrite Γ relative to these bases. More generally we can compose $\langle \Gamma \mid \dots \rangle$ with linear operators $X_a : V_a \rightarrow U_a$, defining a new tensor Γ^X on axes $V_1, \dots, V_\ell$ by

$$(6) \quad \langle \Gamma^X \mid u_1, \dots, u_\ell \rangle := \langle \Gamma \mid X_1 u_1, \dots, X_\ell u_\ell \rangle.$$

3. SPARSITY PATTERNS

Adopting the notational convention above, we can simplify the sparsity pattern condition (8). A *sparsity pattern* for a tensor Γ exhibited by decompositions

$$(7) \quad U_a = X_{a1} \oplus X_{a2} \oplus \cdots \oplus X_{ak_a}, \quad a \in [\ell],$$

is a set $\Delta \subset \prod_a [k_a]$ satisfying

$$(8) \quad (i_1, \dots, i_\ell) \notin \Delta \implies \langle \Gamma \mid X_{1i_1}, \dots, X_{\ell i_\ell} \rangle = 0,$$

where $\langle \Gamma \mid X_{1i_1}, \dots, X_{\ell i_\ell} \rangle$ is the span of all partial evaluations at all $u \in \prod_{a \in [\ell]} X_{ai_a}$.

Writing Γ as a multiarray with respect to bases of the modes that respect the decompositions in (7), the entries of Γ form blocks corresponding to tuples $(i_1, \dots, i_\ell)$ indexing the summands. The tuples in the sparsity pattern Δ correspond to blocks that are permitted to have nonzero entries. Every subset of $[k_1] \times \cdots \times [k_\ell]$ occurs as the sparsity pattern of some tensor, and a single tensor may have several sparsity patterns corresponding to different decompositions of its modes.

Sparsity patterns $\Delta, \Upsilon \subseteq [k_1] \times \cdots \times [k_\ell]$ are considered *equivalent* if, for each $a \in [\ell]$, there is a permutation σ_a of $[k_a]$ such that

$$\Upsilon = \{(\sigma_1(i_1), \dots, \sigma_\ell(i_\ell)) : (i_1, \dots, i_\ell) \in \Delta\}.$$

We can visualize sparsity patterns in multiarray tensors by computing, for each $a \in [\ell]$, contractions on some ordered basis of X_{ai_a} , where $(i_1, \dots, i_\ell) \in \Delta$. As the spaces X_{ai_a} decompose the mode U_a , the contractions U_a can thus be visualized as a square matrix partitioned into k_a columns. This introduces an arbitrary ordering, and our visualizations of the corresponding sparsity patterns depict this order.

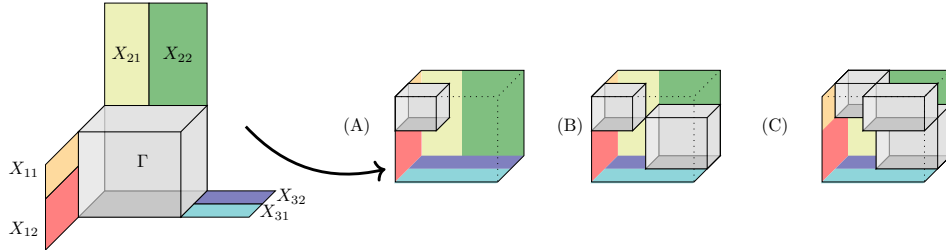


FIGURE 3. Sparsity patterns exhibited by decompositions of arrays. In (A), the pattern is $\Delta = \{(1, 1, 1)\}$; in (B), $\Delta = \{(1, 1, 1), (2, 2, 2)\}$; and in (C), $\Delta = \{(1, 1, 2), (1, 2, 1), (2, 2, 2)\}$.

The granularity of the sparsity patterns we seek is calibrated by the dimensions of the spaces X_{a,i_a} in the decompositions. This allows sparsity patterns to describe a spectrum of tensor phenomena. Figures 3 and 4 illustrate a variety of sparsity patterns for multiarray tensors of valence 3 using shaded regions to indicate nonzero regions.

The objective, given a tensor and a sparsity pattern, is to find bases for the decompositions of the modes that exhibit the sparsity pattern. *In particular, we emphasize that full linear transformations are allowed on each mode; we are not constrained to only working with permutations of the entries in a multiarray.*

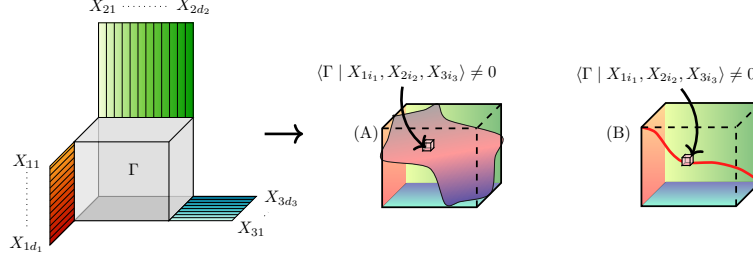


FIGURE 4. Curve and surface sparsity patterns.

4. CHISELS

The definition of sparsity pattern given in the previous section is too broad to be practicable. It includes, for example, all higher-dimensional analogues of Latin square configurations. We focus our attention on families of sparsity patterns that are located on or near the intersection of hyperplanes. For this, we introduce a device we call a *chisel*, which in the basic case is an $m \times \ell$ matrix

$$(9) \quad \mathcal{C} = \begin{bmatrix} C_{11} & \cdots & C_{1\ell} \\ \vdots & & \vdots \\ C_{m1} & \cdots & C_{m\ell} \end{bmatrix}$$

with entries in $\mathbb{K}$. The integer ℓ corresponds to the valence of the tensors upon which the chisel is deployed, and m corresponds to the number of planes involved in the sparsity pattern. Each pair $(\mathcal{C}, \delta)$, where $\mathcal{C}$ is a chisel and

$$(10) \quad \delta = (\delta_1, \dots, \delta_\ell) \in \mathbb{K}^{k_1} \times \cdots \times \mathbb{K}^{k_\ell}$$

for positive integers $k_1, \dots, k_\ell$, defines the sparsity pattern

$$(11) \quad \Delta(\mathcal{C}, \delta) = \left\{ (i_1, \dots, i_\ell) \in \prod_{a=1}^{\ell} [k_a] : \forall j \in [m], \sum_{a=1}^{\ell} C_{ja} \delta_a(i_a) = 0 \right\}.$$

For example, consider the pair

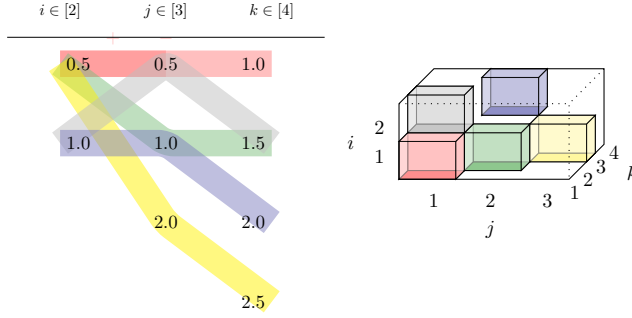
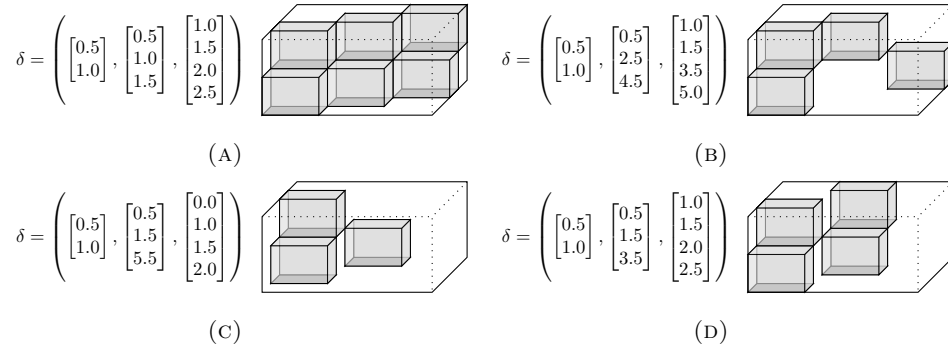
$$(12) \quad \mathcal{C} = \begin{bmatrix} 1 & 1 & -1 \end{bmatrix}, \quad \delta = \left(\begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}, \begin{bmatrix} 0.5 \\ 1.0 \\ 2.0 \end{bmatrix}, \begin{bmatrix} 1.0 \\ 1.5 \\ 2.0 \\ 2.5 \end{bmatrix} \right) \in \mathbb{K}^2 \times \mathbb{K}^3 \times \mathbb{K}^4.$$

Since $1 \cdot \delta_1(1) + 1 \cdot \delta_2(2) + (-1) \cdot \delta_3(2) = 0.5 + 1.0 - 1.5 = 0.0$, we see that $\Delta(\mathcal{C}, \delta)$ contains the tuple $(1, 2, 2)$. In fact,

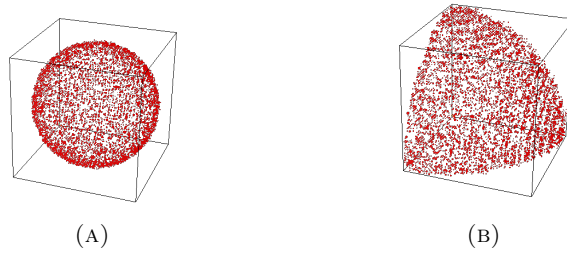
$$\Delta(\mathcal{C}, \delta) = \{(1, 1, 1), (1, 2, 2), (1, 3, 4), (2, 1, 2), (2, 2, 3)\}.$$

Figure 5 plots the possible nonzero values of a tensor with sparsity pattern $\Delta(\mathcal{C}, \delta)$. On the left is an ℓ -uniform ℓ -partite hypergraph capturing the combinations of $\delta_a(i_a)$ that sum to zero, and on the right is a point cloud of a tensor showing the possible regions where the points may congregate.

Our goal, for a given tensor and chisel $\mathcal{C}$, is to determine mode decompositions that exhibit the patterns $\Delta(\mathcal{C}, \delta)$ as sparsity patterns of the tensor. The shapes of sparsity patterns can vary even with the same chisel. For instance, each of the patterns in Figure 6 arise from the chisel $\mathcal{C} = [1, 1, -1]$ for different choices of δ .

FIGURE 5. Sparsity pattern $\Delta(\mathcal{C}, \delta)$ for the $(\mathcal{C}, \delta)$ in equation (12).FIGURE 6. Sparsity patterns $\Delta([1, 1, -1], \delta)$ for some choices of δ .

Sparsity patterns corresponding to some chisel generally bear no resemblance to the hyperplanes defining the chisel. Take for instance the discretized surface of a sphere with radius r and center (c_1, c_2, c_3) , as illustrated in Figure 7(A). The equation of this surface is $(x - c_1)^2 + (y - c_2)^2 + (z - c_3)^2 = r^2$, so if we set $\delta_a(i_a) = (i_a - c_a)^2 - r^2/3$ then $\delta_1(i) + \delta_2(j) + \delta_3(k) = 0$ exactly when $(i - c_1)^2 + (j - c_2)^2 + (k - c_3)^2 = r^2$. Thus, the sparsity pattern $\Delta(\mathcal{C}, \delta)$ for the chisel $\mathcal{C} = [1, 1, 1]$ describes the sphere.

FIGURE 7. Sparsity pattern $\Delta([1, 1, 1], \delta_a(i_a) = (i_a - c_a)^2 - r^2/3)$ together with its sorted level-set form.

The sphere pattern is clearly symmetric, with

$$\delta_a(i_a) = (\pm(i_a - c_a))^2 + r^2/3 = \delta_a(2c_a - i_a).$$

Choosing a permutation σ_a that arranges the sequence of scalars $\delta_a(i_{\sigma(a)})$ in decreasing order, we will group together coordinates with the same δ values. In that order the pattern appears as a discretized octant of the sphere 8 layers thick, as illustrated in Figure 7(B).

One of the design choices in our algorithms is to use such an ordering. Our algorithms are therefore predisposed to locate sparsity patterns of the type depicted in Figure 7(B) rather than in Figure 7(A). Nevertheless, any tensor admitting one of these patterns necessarily admits the other. Thus, one can view our design choice as selecting a canonical order from the possible orders. A notebook replicating this experiment can be found in [8, SphereLab].

5. DERIVATIONS

Chisels have so far been used only to *define* the classes of sparsity patterns we wish to study. In this section we explain how they can also be used to *detect* sparsity patterns in a given tensor. Historically, derivations have proved to be useful when studying products. Through their interpretation as multilinear functions, tensors generalize distributive binary products. It is therefore reasonable to explore invariants for tensors that behave like derivations.

5.1. Derivations associated with chisels. Consider a 3-tensor with $U_1 = U_2 = U_3$. Recall from our notational convention, this could have come from the multiplication $*$: $A \times A \rightarrow A$ of a $\mathbb{K}$ -algebra A . As in [17, Chapter III, Section 6], a *derivation* in this context is a $\mathbb{K}$ -linear function $D : A \rightarrow A$ satisfying

$$\forall a \in A, \forall b \in A, \quad D(a * b) = D(a) * b + a * D(b).$$

Algebras can act on other spaces, leading to more general bilinear products of the form $*$: $A \times B \rightarrow C$. Since the operator D can no longer act in the same way on the three spaces A, B, C , we must generalize the derivation definition accordingly. Denoting those actions by D_A, D_B , and D_C , we get the condition

$$\forall a \in A, \forall b \in A, \quad D_C(a * b) = D_A(a) * b + a * D_B(b).$$

In view of our notational convention, write $*$ as a product $\langle \cdot \cdot \cdot \rangle : A \times B \times C^* \rightarrow \mathbb{K}$, where $\langle a, b, c^\top \rangle := c^\top(a * b)$. We can then recast the derivation condition as follows:

$$\forall a \in A, \forall b \in B, \forall c \in C, \quad \langle D_A(a), b, c \rangle + \langle a, D_B(b), c \rangle - \langle a, b, D_C^*(c) \rangle = 0.$$

This suggests the following general definition.

Definition 5.1. A tuple $(D_1, \dots, D_\ell)$ of operators on the modes $U_1, \dots, U_\ell$ of a tensor Γ is a $\mathcal{C}$ -derivation of Γ if it satisfies the linear system

$$\begin{aligned} \mathcal{C}_{11} \langle \Gamma \mid D_1 u_1, \dots, u_\ell \rangle &+ \dots + \mathcal{C}_{1\ell} \langle \Gamma \mid u_1, \dots, D_\ell u_\ell \rangle &= 0 \\ \vdots & & \vdots \\ \mathcal{C}_{m1} \langle \Gamma \mid D_1 u_1, \dots, u_\ell \rangle &+ \dots + \mathcal{C}_{m\ell} \langle \Gamma \mid u_1, \dots, D_\ell u_\ell \rangle &= 0 \end{aligned} \tag{13}$$

for all $(u_1, \dots, u_\ell) \in U_1 \times \dots \times U_\ell$. Define

$$\text{Der}(\mathcal{C}, \Gamma) = \{ (D_1, \dots, D_\ell) \text{ satisfying system (13)} \} \tag{14}$$

to be the $\mathbb{K}$ -space of all $\mathcal{C}$ -derivations of Γ .

To help digest this key definition, we describe the space of derivations of a tensor coming from a finite-dimensional algebra.

Example 5.2. Consider the polynomial algebra $\mathbb{K}[x]/(x^d-1)$. It has a multiplication table relative to the basis $\{1, x, x^2, \dots, x^{d-1}\}$ with relations $x^i * x^j = x^{i+j}$ if $i+j < d$ and $x^i * x^j = x^{i+j-d}$ if $i+j \geq d$, and is therefore a 3-tensor represented as an array $\Gamma \in \mathbb{K}^{d \times d \times d}$, where

$$\Gamma[\cdot, \cdot, 1] = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 1 \\ \vdots & \ddots & \ddots & 1 & 0 \\ \vdots & 0 & \ddots & \vdots & \vdots \\ 0 & 1 & 0 & \cdots & 0 \end{bmatrix} \quad \Gamma[\cdot, \cdot, 2] = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & \vdots & 0 & \cdots & 1 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 1 & \cdots & 0 \end{bmatrix} \cdots$$

Let $\mathbb{A}$ consist of all matrices of the form

$$\begin{bmatrix} \lambda_0 & \lambda_1 & \cdots & \lambda_{d-1} \\ \lambda_{d-1} & \lambda_0 & \lambda_1 & \cdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \lambda_2 & & \lambda_{d-1} & \lambda_0 & \lambda_1 \\ \lambda_1 & \lambda_2 & \cdots & \lambda_{d-1} & \lambda_0 \end{bmatrix}, \quad \lambda_i \in \mathbb{K}.$$

Solving for D_1, D_2, D_3 in the linear system (13) with $\mathcal{C} = [1, 1, -1]$, we get

$$\text{Der}([1, 1, -1], \Gamma) = \{(X, Y, X + Y) \mid X, Y \in \mathbb{A}\}.$$

Remark 5.3. Suppose, for some $a \in [\ell]$, that column a of $\mathcal{C}$ consists entirely of zeros. Then the derivation condition imposes no constraint on the operator D_a . In that case we say that $\mathcal{C}$ is *not engaged* on mode a and impose the condition $D_a = 0$.

5.2. Sparsity patterns produce derivations. If we have a decomposition that exhibits a sparsity pattern for a tensor, then we can always find derivations:

Proposition 5.4. *Let Γ be a ℓ -tensor, $\mathcal{C}$ a chisel, and $\delta \in \mathbb{K}^{k_1} \times \cdots \times \mathbb{K}^{k_\ell}$. Suppose we have decompositions $U_a = X_{a1} \oplus \cdots \oplus X_{ak_a}$ of the modes that exhibit the sparsity pattern $\Delta(\mathcal{C}, \delta)$ for Γ . For $a \in [\ell]$, if D_a is the linear operator with δ_{ai_a} -eigenspaces X_{ai_a} , then $D = (D_1, \dots, D_\ell)$ is a $\mathcal{C}$ -derivation of Γ .*

Proof. For each $a \in [\ell]$ and $i_a \in [k_a]$, let x_{ai_a} be vectors in X_{ai_a} , which by construction are δ_{ai_a} -eigenvectors of D_a . Hence, for each $1 \leq j \leq m$, we have

$$\begin{aligned} \mathcal{C}_{j1} \langle \Gamma \mid D_1 x_{1i_1}, x_{2i_2}, \dots, x_{\ell i_\ell} \rangle &+ \cdots + \mathcal{C}_{j\ell} \langle \Gamma \mid x_{1i_1}, \dots, x_{\ell-1 i_{\ell-1}}, D_\ell x_{\ell i_\ell} \rangle \\ &= \mathcal{C}_{j1} \delta_{1i_1} \langle \Gamma \mid x_{1i_1}, \dots, x_{\ell i_\ell} \rangle + \cdots + \mathcal{C}_{j\ell} \delta_{\ell i_\ell} \langle \Gamma \mid x_{1i_1}, \dots, x_{\ell i_\ell} \rangle \\ &= \left(\sum_{a=1}^{\ell} \mathcal{C}_{ja} \delta_{ai_a} \right) \langle \Gamma \mid x_{1i_1}, \dots, x_{\ell i_\ell} \rangle. \end{aligned}$$

Equation (13) is satisfied for $u_a = x_{ai_a}$ if $\langle \Gamma \mid x_{1i_1}, \dots, x_{\ell i_\ell} \rangle = 0$. If not, since the decomposition exhibits the sparsity pattern $\Delta(\mathcal{C}, \delta)$, we have $\sum_{a=1}^{\ell} \mathcal{C}_{ja} \delta_{ai_a} = 0$, so equation (13) is still satisfied for $u_a = x_{ai_a}$. As the spaces X_{ai_a} decompose the modes of Γ , the union of any choice of bases for them span the U_a . It follows that $(D_1, \dots, D_\ell)$ satisfies condition (13), and hence is a $\mathcal{C}$ -derivation. $\square$

5.3. Derivations reveal sparsity patterns. We have just seen that sparsity patterns in tensors ensure the existence of derivations. Conversely, derivations can be leveraged to discover sparsity patterns in tensors.

Theorem 5.5. *Let $(D_1, \dots, D_\ell)$ be a diagonalizable $\mathcal{C}$ -derivation of a tensor Γ for some chisel $\mathcal{C}$. For each $a \in [\ell]$, let $X_{a,1}, \dots, X_{a,k_a}$ be the eigenspaces of D_a with eigenvalues $\delta_a(1), \dots, \delta_a(k_a)$. This eigenspace decomposition of the modes exhibits the sparsity pattern $\Delta(\mathcal{C}, (\delta_1, \dots, \delta_\ell))$ of Γ .*

Proof. For $a \in [\ell]$, let X_{a,i_a} be the δ_{a,i_a} -eigenspace of the diagonalizable operator D_a . Since $(D_1, \dots, D_\ell)$ is a $\mathcal{C}$ -derivation, for each $j \in [m]$ and $x_{a,i_a} \in X_{a,i_a}$,

$$\begin{aligned} 0 &= \mathcal{C}_{j1} \langle \Gamma \mid D_1 x_{1,i_1}, \dots, x_{\ell,i_\ell} \rangle + \dots + \mathcal{C}_{j\ell} \langle \Gamma \mid x_{1,i_1}, \dots, D_\ell x_{\ell,i_\ell} \rangle \\ &= \mathcal{C}_{j1} \langle \Gamma \mid \delta_{1,i_1} x_{1,i_1}, \dots, x_{\ell,i_\ell} \rangle + \dots + \mathcal{C}_{j\ell} \langle \Gamma \mid x_{1,i_1}, \dots, \delta_{\ell,i_\ell} x_{\ell,i_\ell} \rangle \\ &= \left(\sum_{a=1}^{\ell} \mathcal{C}_{ja} \delta_{a,i_a} \right) \langle \Gamma \mid x_{1,i_1}, \dots, x_{\ell,i_\ell} \rangle. \end{aligned}$$

It follows that

$$\langle \Gamma \mid X_{1,i_1}, \dots, X_{\ell,i_\ell} \rangle \neq 0 \implies (i_1, \dots, i_\ell) \in \Delta(\mathcal{C}, \delta),$$

so the decompositions exhibit the sparsity pattern $\Delta(\mathcal{C}, \delta)$ in Γ . $\square$

Remark 5.6. It is natural to wonder what happens if we have a derivation for which not every D_a is diagonalizable. This happens, for instance, when some D_a is nilpotent or a rotation on U_a . A version of Theorem 5.5 holds for generalized eigenspaces, where each operator D_a can be decomposed into Jordan blocks. In particular, Theorem 5.5 holds for *any* derivation when $\mathbb{K}$ is the complex field $\mathbb{C}$.

5.4. Scalar derivations reveal nothing. Let $(D_1, \dots, D_\ell)$ be a derivation of a tensor Γ . If the eigenspace decomposition of some D_a consists of the single space U_a , then D_a has a single eigenvalue δ_a , so $D_a = \delta_a 1_{U_a}$. If this is the case for every $a \in [\ell]$, we call $(D_1, \dots, D_\ell)$ a *scalar derivation* of Γ .

Notice that if $\delta \in \mathbb{K}^\ell$ is a row vector with $\mathcal{C}\delta^* = 0$, then $(\delta_1 1_{U_1}, \dots, \delta_\ell 1_{U_\ell})$ is a scalar derivation of any tensor Γ . Hence, the dimension of $\text{Der}(\mathcal{C}, \Gamma)$ is at least the dimension of the sparsity space of $\mathcal{C}$. If $(D_1, \dots, D_\ell)$ is a scalar derivation, then the sparsity pattern $\Delta(\mathcal{C}, \delta) = \{(1, \dots, 1)\}$ is trivial. Thus, scalar derivations tell us nothing about the existence of nontrivial sparsity patterns in a given tensor, so we disregard them.

6. CHISELING

Theorem 5.5 can be used as the foundation of practical algorithms to detect sparsity patterns in tensors. In this section we present two such algorithms. The first, Algorithm 1, is an elementary field agnostic version based directly on Theorem 5.5. The second, Algorithm 2, is a more practical heuristic version designed for use on real data. Our Julia implementation [8] is based on Algorithm 2.

6.1. Basic chiseling. Algorithm 1 takes as input a tensor Γ and a chisel $\mathcal{C}$ and returns a decomposition of the modes of Γ exhibiting a sparsity pattern $\Delta(\mathcal{C}, \delta)$ for some non-scalar derivation $\delta \in \text{Der}(\mathcal{C}, \Gamma)$, if such exists.

Discussion: When working over fields with exact operations, such as finite fields or the rationals, there are several ways to accomplish Lines 1 and 2. The system (13)

Algorithm 1 (chiseling in tensors — a basic version)**Input:** a tensor Γ , and a chisel $\mathcal{C}$.**Output:** decompositions $U_a = X_{a,1} \oplus \cdots \oplus X_{a,k_a}$ of the modes of Γ exhibiting a nontrivial sparsity pattern $\Delta(\mathcal{C}, \delta)$ of Γ , for some δ .

- 1: Find a non-scalar solution $(D_1, \dots, D_\ell)$ to the linear system (13), where each D_a is diagonalizable.
- 2: For each a , compute the eigenvalues $\delta_a = (\delta_{a,i_a} : i_a \in [k_a])$ of D_a and the corresponding eigenspace decomposition $U_a = X_{a,1} \oplus \cdots \oplus X_{a,k_a}$.
- 3: **return** bases for the spaces X_{a,i_a} and the sparsity pattern $\Delta(\mathcal{C}, (\delta_a : a \in [\ell]))$.

is linear so we can solve it using established techniques such as least-squares, LU decomposition, and pseudo-inverses. Algorithms for computing the eigenspace decompositions in Line 2 can also be found in standard linear algebra packages. There are issues even in the exact field setting—we mention two of them.

First, how can we avoid unnecessary overhead in solving (13)? The default approach solves a linear system with $e = (d_1 \cdots d_\ell)$ equations in $v = d_1^2 \times \cdots \times d_\ell^2$ variables. A solution to this large and vastly over-determined system can proceed by sampling $(1 + \varepsilon)v$ equations to solve an approximately square system M . Furthermore, each equation uses $O(\sqrt{v})$ variables (far less if Γ is a sparse array), so mat-vecs Mv can be evaluated using $O(v^{1.5})$ field operations. An iterative solver can find a solution to M in $O(v^{2.5}) \subset O((\ell d)^5)$, where $d = \max\{d_1, \dots, d_\ell\}$ operations. Since this system is nearly square, with high probability it has few solutions, so any solution to $Mv = 0$ is likely to be a solution to the entire system.

The complexity is therefore $O((\ell d)^5)$, which is expensive. In fact, absent additional engineering, this is quadratically worse than using HOSVD [14]. It would be better to leave the system as it is given and use tensor methods to solve it. A project to develop such a solver is currently underway. Note, however, that even reading the data in a dense $d_1 \times \cdots \times d_\ell$ array Γ requires $O(d^\ell)$ operations.

The second issue is how to select a non-scalar derivation D from the solution space in a way that produces the best possible results? We comment further on this issue after we present Algorithm 2 below.

6.2. Chiseling with SVDs. Algorithm 1 is more or less field agnostic, but as stated it is clearly inadequate for use with practical models of the real and complex numbers. In these settings, one can only target approximate solutions to (13), and one must deploy robust floating point algorithms. For the remainder of the section, we assume that $\mathbb{K}$ is either the real field $\mathbb{R}$ or the complex field $\mathbb{C}$. Algorithm 2 is a heuristic version of Algorithm 1 that works well in these settings.

Discussion: The SVD of N produces approximate solutions to the nullspace by inspecting the singular values closest to 0. One key advantage of SVD is that the singular values are ordered. This provides a canonical choice of vector from which to build a ‘good’ derivation D in Line 1 of Algorithm 1, as we now explain.

Recall that each chisel $\mathcal{C}$ has its own nullspace of dimension e , and the vectors in this nullspace give rise to scalar derivations. As these involve scalar matrices, they arise as solutions to (13) with very little error—they correspond in the SVD to the singular values closest to 0. We therefore skip to the $(e+1)^{\text{st}}$ smallest singular value, σ_{e+1} , and regard σ_{e+1} being close to 0 as a proxy for Γ having a sparsity pattern

Algorithm 2 (chiseling in tensors — a heuristic version)

Input: a tensor Γ , and a chisel $\mathcal{C}$.

Output: decompositions $U_a = X_{a,1} \oplus \cdots \oplus X_{a,k_a}$ of the modes of Γ exhibiting a sparsity pattern $\Delta(\mathcal{C}, \delta)$ for some non-scalar $\mathcal{C}$ -derivation δ of Γ .

- 1: Build the matrix N whose nullspace parameterizes (13).
 - 2: Compute the dimension e of the nullspace of $\mathcal{C}$.
 - 3: Computing the SVD of N , locate the $(e+1)^{\text{st}}$ smallest singular value, σ_{e+1} .
 - 4: If σ_{e+1} is close to 0, use the corresponding singular vector to build matrices $(D_1, \dots, D_\ell)$ that approximately satisfy (13). Otherwise report that Γ does not conform to any sparsity pattern determined by $\mathcal{C}$.
 - 5: For $a \in [\ell]$, compute invertible X_a such that $D_a X_a = X_a \text{diag}(\delta_{a,1}, \dots, \delta_{a,k_a})$.
 - 6: **return** $(X_a : a \in [\ell])$ and the sparsity pattern $\Delta(\mathcal{C}, (\delta_a : a \in [\ell]))$.
-

of the type detected by the chisel $\mathcal{C}$. The corresponding singular vector produces $(D_1, \dots, D_\ell)$ that approximates a non-trivial $\mathcal{C}$ -derivation. Thus, proceeding in this way removes the search steps in Algorithm 1.

Remark 6.1. Algorithms 1 and 2 only use diagonalizable operators. This is usually the case in practical settings, but one can adapt the technique to handle generalized eigenspaces. Indeed, there are situations where it is interesting to consider nilpotent derivations, but we only focus on the semisimple case in this article.

Remark 6.2. Based on experiments with the Julia implementation, our methods tolerate a certain amount of noise in the data. Recently, Nick Vannieuwenhoven has conducted a more robust analysis of variations of Algorithm 2 under noise [24].

7. CHISELING TOOLS FOR 3-TENSORS

A key feature of Algorithms 1 and 2 is the control they afford users to search for the type of structure they wish to find in their tensor data—or, more likely, that they believe exists within their data. The control comes from the ability to choose the chisel matrix $\mathcal{C}$, and in this section we describe several choices.

In Sections 7.1–7.2 and 7.3 we introduce three chisels we have found particularly useful in our experiments. We describe the sparsity patterns that each chisel $\mathcal{C}$ reveals, and give demonstrations using our Julia implementation of Algorithm 2 on synthetically created tensors. In Section 7.4 we explain why these chisels reveal the corresponding sparsity patterns in tensors, and in Section 7.5 we show how the familiar Tucker decomposition and (special cases of) HOSVD can be carried out by chiseling. For reasons of visualization we confine our attention to 3-tensors throughout this section, but the properties we discuss have natural generalizations to higher valence tensors.

7.1. The universal chisel. We first introduce the chisel that serves as a default chisel—for reasons explained in Proposition 9.1, we call it the *universal chisel*:

$$(15) \quad \mathcal{C}_{\text{uni}} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}.$$

The solution space $\text{Der}(\mathcal{C}_{\text{uni}}, \Gamma)$ of (13) using chisel $\mathcal{C}_{\text{uni}}$ forms a Lie algebra, where the Lie bracket is the commutator map [7]. We use this fact in the next

section to derive our uniqueness results. The subspace of scalar $\mathcal{C}_{\text{uni}}$ -derivations is given by

$$(16) \quad \text{scl}(\mathcal{C}_{\text{uni}}) = \{ (\lambda_1 I_{d_1}, \lambda_2 I_{d_2}, \lambda_3 I_{d_3}) : \lambda_1 + \lambda_2 + \lambda_3 = 0 \},$$

which has dimension 2. Hence, when using $\mathcal{C}_{\text{uni}}$ in Algorithm 2, we focus on the third smallest singular value of the matrix N in Line 3.

The universal chisel $\mathcal{C}_{\text{uni}}$ was used to reveal the sparsity patterns depicted in Figure 1(C) and (D). Experiments that recover images similar to these are described in Figures 8 and 9, respectively.

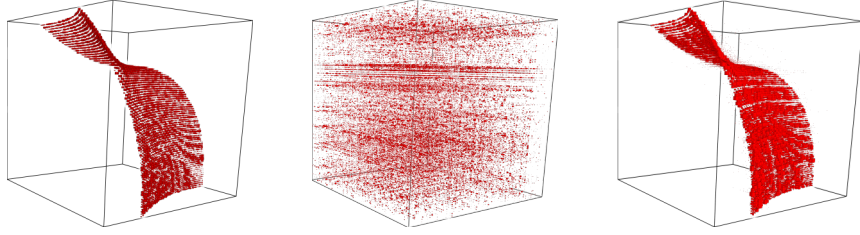


FIGURE 8. The $81 \times 81 \times 81$ tensor on the left has support constrained near a surface. The tensor was then scrambled with random basis changes to produce the middle tensor. It was then passed to Algorithm 2 with chisel $\mathcal{C}_{\text{uni}}$, and it returned the tensor on the right.

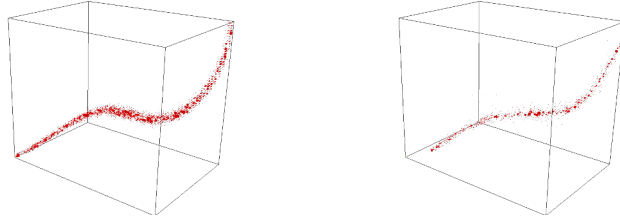


FIGURE 9. The $(40 \times 60 \times 70)$ -array on the left was constructed to have support near a curve. The array was again scrambled (image omitted) as in Figure 8. The scrambled array was then passed to our algorithm, which reconstructed the array on the right.

Recall from Section 4 that [8, SphereLab] is a notebook that demonstrates how to use our algorithm to recover sparsity patterns for data congregated near the surface of a sphere. It uses the universal chisel to do this and can be adapted to experiment with other “continuous” sparsity patterns.

7.2. Adjoint chisels. The tensor depicted in Figure 1(B) is a decomposition with respect to the front face. Specifically, it exhibits a sparsity pattern for decompositions of just axes 1 and 2 into three nontrivial subspaces. Methods to obtain decompositions of this type have been developed by other researchers [1, 9–11]. These patterns can be detected by Algorithm 2 using an *adjoint chisel*.

There is one adjoint chisel for each (unordered) pair of modes that define the face in question. Thus, for a tensor with ℓ modes, there are $\binom{\ell}{2}$ adjoint chisels, each consisting of a single row. The adjoint chisel for Figure 1(B) is

$$(17) \quad \mathcal{C}_{\text{adj}} = \begin{bmatrix} 1 & -1 & 0 \end{bmatrix}.$$

The subspace of scalar $\mathcal{C}_{\text{adj}}$ -derivations is given by

$$(18) \quad \text{scl}(\mathcal{C}_{\text{adj}}) = \{ (\lambda I_{d_1}, \lambda I_{d_2}, 0_{d_3}) : \lambda \in \mathbb{K} \},$$

and is therefore 1-dimensional. Hence, when using $\mathcal{C}_{\text{adj}}$ in Algorithm 2, we focus on the second smallest singular value of the matrix N in Line 3.

As one of the modes is disengaged in the adjoint chisels, it is not surprising that there is no restriction on the data in the tensor along that mode. In fact, the adjoint chisels reveal ‘face block’ structure along the face corresponding to the two active modes. Figure 10 shows the outcome of the now familiar experiment.



FIGURE 10. The $(50 \times 60 \times 70)$ -array on the right was reconstructed by our algorithm using the $\mathcal{C}_{\text{adj}}$ chisel after we scrambled the synthetically constructed tensor on the left.

An experiment similar to this one (using an adjoint chisel) can be replicated using the notebook [8, FaceBlockLab].

7.3. The centroid chisel. If we discover face block decompositions in two faces simultaneously we get diagonal blocks. In particular, we can determine whether a tensor decomposes into diagonal blocks using the single chisel

$$(19) \quad \mathcal{C}_{\text{cen}} = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \end{bmatrix}.$$

For a given tensor Γ , the space $\text{Cen}(\Gamma) := \text{Der}(\mathcal{C}_{\text{cen}}, \Gamma)$ has the structure of a commutative ring called the *centroid* of Γ , where the product is just matrix multiplication in each coordinate. This ring and its properties have been studied in numerous ways [2, 6, 19, 25]. Figure 11 shows another experiment to illustrate the recovery of diagonal blocks.

An experiment similar to this one (using the centroid chisel) can be replicated using the notebook [8, BlockLab].

7.4. Explaining the outcomes. Why do the chisels we have highlighted in this section reveal the sparsity patterns we have seen in these experiments? Let us start with the adjoint chisel $\mathcal{C}_{\text{adj}} = [0, 1, -1]$ and a $\mathcal{C}_{\text{adj}}$ -derivation (D_1, D_2, D_3) , so that

$$0 \cdot \langle \Gamma \mid D_1 u_1, u_2, u_3 \rangle + 1 \cdot \langle \Gamma \mid u_1, D_2 u_2, u_3 \rangle + (-1) \cdot \langle \Gamma \mid u_1, u_2, D_3 u_3 \rangle = 0.$$

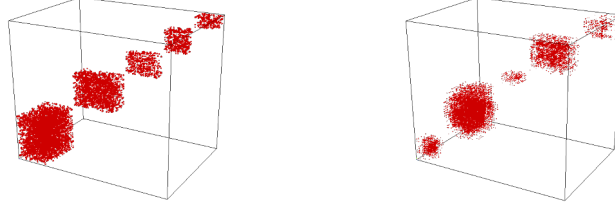


FIGURE 11. The array with prescribed block structure on the left was scrambled and then passed as input to Algorithm 2 with the $\mathcal{C}_{\text{cen}}$ chisel; the array on the right is the output.

The first mode is not engaged so we insist that $D_1 = 0$ (Remark 5.3). We can use partial evaluation to remove mode 1 from consideration:

$$u * v := \langle \Gamma | -, u, v \rangle.$$

Putting $X := D_3$ and $X^\circ := D_2$, the $\mathcal{C}_{\text{adj}}$ -derivation condition for $(0, X^\circ, X)$ becomes the familiar adjoint condition

$$(X^\circ u) * v = u * (Xv).$$

from linear algebra (hence the name) expressed for the bilinear map $*$: $U_2 \times U_3 \rightarrow U_1^*$ and, as in linear algebra, this means

$$(20) \quad (\text{Null } X^\circ) * (Xv) = X^\circ(\text{Null } X^\circ) * v = 0.$$

Because $(0, \lambda I, \lambda I)$ is a $\mathcal{C}_{\text{adj}}$ -derivation, it follows that $(0, X^\circ - \lambda I, X - \lambda I)$ is also a $\mathcal{C}_{\text{adj}}$ -derivation. By condition (20), we know that the null spaces and images of these matrices are orthogonal. Thus, to find and diagonalize a $[0, 1, -1]$ -derivation of Γ is to find an orthogonal decomposition on the $\{2, 3\}$ -face of Γ .

Next consider the centroid chisel $\mathcal{C}_{\text{cen}}$, and note that the space of $\mathcal{C}_{\text{cen}}$ -derivations is the intersection of adjoint derivations on two or more faces of Γ . Thus the sparsity patterns for $\mathcal{C}_{\text{cen}}$ coincide with blocks along the diagonal of the array.

The surface sparsity patterns revealed by the universal chisel $\mathcal{C}_{\text{uni}}$ are the most surprising, but we have already seen in the discussion about the sphere in Section 4 how these continuous cases are indeed sparsity patterns for the universal chisel.

7.5. Tucker and Higher-Order Singular Value Decompositions. We mentioned in the introduction that some known tensor decompositions are special cases of chiseling. We explain how these decompositions are recovered by our methods.

Tensor decompositions of form depicted in Figure 3(A) are known as Tucker decompositions [23], and many efficient algorithms have been developed to find them [20]. Proposition 8.1 describes the chisels needed to carry out Tucker decompositions. In particular, a full Tucker decomposition may be carried out using the identity matrix as a chisel, since condition (13) then becomes

$$\forall a \in [\ell], \quad \langle \Gamma | u_1, \dots, D_a u_a, \dots, u_\ell \rangle = 0.$$

In words, the image of D_a lies in the radical of $\langle \Gamma | - \rangle$, and the a -th Tucker decomposition is precisely that of the image and kernel of D_a .

Focusing attention on a single axis a of a multilinear map $\gamma : U_1 \times \cdots \times U_\ell \rightarrow \mathbb{K}$ is functionally the same as treating γ as a linear map

$$L_a : U_a \rightarrow (U_1 \otimes \cdots \otimes U_{a-1} \otimes U_{a+1} \otimes \cdots \otimes U_\ell)^*.$$

The nullspace of L_a is the a -th radical and the derivation condition $L_a D_a = 0$ identifies this nullspace. More generally the singular values of L_a are known as higher-order singular values. Our methods find D_a such that $L_a D_a \approx 0$, so the D_a eigenspaces coincide with the lowest HOSVD components of the tensor.

We reiterate that we are not proposing that our algorithm is a replacement for the highly optimized Tucker and HOSVD decomposition algorithms, only that they can be viewed as special cases of chiseling.

8. THE THEORY OF CHISELS

The previous section focused on a small collection of chisels designed to reveal specific types of sparsity patterns. These are also the chisels we have found most useful in our experience. However, there is more to say about chiseling in general, and our methods have the capability to work with chisels in more refined ways than the basic algorithms describe. In this section we elaborate on tensor chiseling.

8.1. Ordering chisels and Derivations. One way to begin organizing a study of chisels is to consider partial orders. Suppose we have two chisels $\mathcal{C}$ and $\mathcal{C}'$, each with ℓ columns and we say define $\mathcal{C} \leq \mathcal{C}'$ if every row of $\mathcal{C}$ is a linear combination of the rows of $\mathcal{C}'$. For this to be a partial order we agree to say that $\mathcal{C} \equiv \mathcal{C}'$ as chisels if $\mathcal{C} \leq \mathcal{C}'$ and $\mathcal{C}' \leq \mathcal{C}$, tht is, they have the same row-span.

Now given a tensor Γ and a $\mathcal{C}'$ -derivation $D \in \text{Der}(\mathcal{C}', \Gamma)$, we can use the linear combinations of the equations (13) to fashion the equations of a $\mathcal{C}$ -derivation. Thus:

$$(21) \quad \mathcal{C} \leq \mathcal{C}' \quad \Rightarrow \quad \text{Der}(\mathcal{C}, \Gamma) \supseteq \text{Der}(\mathcal{C}', \Gamma)$$

This argument also indicates that it is the row-span of $\mathcal{C}$, not an fixed matrix, which defines $\mathcal{C}$ -derivations.

As with any linear equation, we learn more about equation (13) by consider its dual. First we fix a set of potential derivations: $\Omega \subseteq \mathbb{K}^{d_1 \times d_1} \times \cdots \times \mathbb{K}^{d_\ell \times d_\ell}$. Given Γ and Ω , we now have linear equations specifying the coefficients C_{ij} in (13). Put

$$\mathcal{C}s(\Gamma, \Omega) := \{\mathcal{C} \in \mathbb{K}^{c \times \ell} \mid \text{equation (13) holds for all } (D_1, \dots, D_\ell) \in \Omega\}.$$

Evidently, $\Omega \subseteq \Omega'$ implies $\mathcal{C}s(\Gamma, \Omega) \supseteq \mathcal{C}s(\Gamma, \Omega')$ and from duality we have the following propety as well.

$$\mathcal{C} \subseteq \mathcal{C}s(\Gamma, \Omega) \iff \Omega \subseteq \text{Der}(\mathcal{C}, \Gamma)$$

By composing $\text{Der}(-, \Gamma)$ and $\mathcal{C}s(\Gamma, -)$ we obtain closure operators on the spaces of chisels and derivations.

$$[\mathcal{C}]_\Gamma := L(\Gamma, \text{Der}(\mathcal{C}, \Gamma)) \quad [\Omega]_\Gamma := \text{Der}(L(\Gamma, \Omega), \Gamma).$$

The chisels and derivations that are informative to an individual tensor are those which are closed. A fuller account of such orders and closures is given in [5, 16].

8.2. A classification of 3-chisels. We just observed that chisels are determined only up to their row-span, but there are a few more subtle equivalences that we can exploit to reduce the number of chisels we need to consider. Suppose rescale the columns of $\mathcal{C}$ by multiplying with an invertible diagonal matrix $\Sigma = \text{Diag}(\sigma_1, \dots, \sigma_\ell)$. Then every $\mathcal{C}$ -derivation for $\mathcal{C}$ -derivation $(D_1, \dots, D_\ell)$ affords a $\mathcal{C}\Sigma$ -derivation $(\sigma_1 D_1, \dots, \sigma_\ell D_\ell)$ and vice-versa. If a $\mathcal{C}$ -derivation $(D_1, \dots, D_\ell)$ detects a pattern $\Delta(\mathcal{C}, \delta)$, then the rescaled derivation $(\sigma_1 D_1, \dots, \sigma_\ell D_\ell)$ detects the pattern $\Delta(\mathcal{C}\Sigma, \Sigma^{-1}\delta)$. So for the purpose of detecting patterns it is sufficient to work with full row rank matrices $\mathcal{C}$ up to the transformation $M\mathcal{C}\Sigma$ for invertible M and diagonal invertible Σ . In fact, it is typically reasonable to reindex the modes of a tensor and so we may add a permutation matrix to Σ to further simplify the classification of chisels.

For 2-tensors (linear maps) the nonzero chisels are $[1, 1]$, $[1, 0]$, and $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ up to this equivalence. The next result gives the classification of chisels for 3-tensors.

Proposition 8.1. *A nonzero 3-chisel is equivalent to exactly one of the following:*

$$\begin{array}{ll}
 \text{(Third Tucker chisel)} & \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \text{(Centroid chisel)} & \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \\
 \text{(Second Tucker chisel)} & \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \text{(Adjoint chisel)} & \begin{bmatrix} 0 & 1 & 1 \end{bmatrix} \\
 \text{(First Tucker chisel)} & \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} & \text{(Universal chisel)} & \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \\
 & & \text{(First Tucker + Adjoint chisel)} & \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}
 \end{array}$$

Proof. First, compute the reduced row echelon form of $\mathcal{C}$ and remove all zero rows. Next, permute the columns so that any zero columns appear to the left. The resulting matrices not explicitly given in the classification have the form

$$\begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix}.$$

If $ab = 0$, this matrix is equivalent to $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$. If $ab \neq 0$, the matrix is equivalent to

$$\begin{bmatrix} 1/a & 0 \\ 0 & 1/b \end{bmatrix} \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix} \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

This completes the classification. $\square$

If we use the 3-valent tensor of rectangular matrix multiplication $\mathbb{K}^{a \times b} \times \mathbb{K}^{b \times c} \rightarrow \mathbb{K}^{a \times c}$ then a calculation shows that the centroid chisel recovers a 1-dimensional space of derivations (the field), the adjoint chisels $[1, 1, 0]$, $[1, 0, 1]$, and $[0, 1, 1]$ each recover spaces $\mathbb{K}^{a \times a}$, $\mathbb{K}^{b \times b}$, and $\mathbb{K}^{c \times c}$, respectively. The universal chisel $[1, 1, 1]$ recovers the all of these as one space $\mathbb{K}^{a^2+b^2+c^2-1}$. In particular, each of these chisels (in the sense of Section 8.1) is closed for this tensor. By defining degenerate products like $\begin{bmatrix} * & * \\ A & * \end{bmatrix} \begin{bmatrix} * & B \\ * & * \end{bmatrix} := \begin{bmatrix} AB & 0 \\ 0 & 0 \end{bmatrix}$, we can prove likewise for the Tucker chisels.

8.3. Higher valence chisels. It suffices, when working with tensors of valence ℓ , to consider just the reduced row echelon forms of the $m \times \ell$ chisel matrices $\mathcal{C}$:

$$\mathcal{C} = \begin{bmatrix} I_r & \mathcal{C}' \\ 0 & 0 \end{bmatrix}.$$

Hence, there are at most $r \times (\ell - r)$ parameters to describe chisels of rank r , corresponding to the different choices of matrix $\mathcal{C}'$. Furthermore, if Σ is an $r \times r$ diagonal matrix, and Σ' a diagonal $(\ell - r) \times (\ell - r)$ matrix, then

$$\begin{bmatrix} \Sigma^{-1} & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} I_r & \mathcal{C}' \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \Sigma & 0 \\ 0 & \Sigma' \end{bmatrix} = \begin{bmatrix} I_r & \Sigma^{-1} \mathcal{C}' \Sigma' \\ 0 & 0 \end{bmatrix}.$$

Hence, up to our equivalence, there are $(r - 1)(\ell - r - 1)$ free parameters in $\mathcal{C}'$, which is positive when $1 < r < \ell - 1$. Thus, when $\ell \geq 4$, the number of inequivalent chisels is at least the size of the field $\mathbb{K}$, which may be infinite or uncountable.

For instance, if we define for $a \in \mathbb{K} \setminus \{0\}$ the family of 4-chisels

$$\mathcal{C}(a) = \begin{bmatrix} 1 & 0 & 1 & a \\ 0 & 1 & 1 & 1 \end{bmatrix},$$

then $\mathcal{C}(a)$ is equivalent to $\mathcal{C}(b)$ only if $a = b$. Hence, over infinite fields there are infinitely many non-equivalent chisels of valence $\ell > 3$.

8.4. Chiseling variations. Finally, often we have reason to restrict operators to special classes, such as orthogonal or unitary transformations. To accomodate for this we could hope to work with $\mathcal{C}$ -derivations $(D_1, \dots, D_\ell)$ that are symmetric or Hermitian as then their eigendecompositions yield orthogonal and unitary transformations. In most situations chisels will not be enough to force operators to be symmetric or Hermitian, but we can simply intersect derivations with these subspaces creating a more constrained class of derivations. We can also consider symmetry of another sort such as symmetry in the original tensor. For a group G of permutations we can apply symmetry to operators by taking the fixed points. All this motivates working with the following generalization of derivations.

Given a space $\Omega \leq \mathbb{K}^{d_1 \times d_1} \times \dots \times \mathbb{K}^{d_\ell \times d_\ell}$ and a chisel $\mathcal{C} \in \mathbb{K}^{c \times \ell}$, we can define

$$\text{Der}_\Omega(\mathcal{C}, \Gamma) := \text{Der}(\mathcal{C}, \Gamma) \cap \Omega.$$

Going forward when we speak of chiseling and derivations we mean to include such variations, as well as variations that acknowledge possible duals spaces amongst the modes as we discussed in the introduction.

8.5. The algebraic structure $\text{Der}_\Omega(\Gamma, \mathcal{C})$. Singular and eigenvalue theories possess uniqueness results that ensure reproducibility. On first inspection chiseling may appear to lack any such uniqueness guarantees. For instance, it may concern the reader that we are happy to use any (nonscalar) derivation δ in $\text{Der}(\mathcal{C}, \Gamma)$ we can get our hands on. However, the sets $\text{Der}(\mathcal{C}, \Gamma)$ generally possess an algebraic structure that explains why generic choices produce the same sparsity patterns.

For example, if a chisel $\mathcal{C}$ consists entirely of 0's and 1's, then $\text{Der}(\mathcal{C}, \Gamma)$ is closed under the Lie algebra product $[D, E] := (D_a E_a - E_a D_a \mid 0 \leq a \leq \ell)$. The same is true of an arbitrary $\mathcal{C}$ in projective coordinates (where u_a in (13) is replaced by $\mathbb{K}u_a$). Derivations for $(\Omega, \mathcal{C})$ pairs can admit algebraic structures, such as associative $*$ -algebras [5], Jordan algebras [22, 26, 27], or commutative rings [2, 6, 19, 25]. Conditions on the possible algebraic products are provided in [16, Theorem D].

I changed D's to Σ for diagonal as D is derivation already

9. UNIVERSALITY & UNIQUENESS

We have alluded throughout to uniqueness properties of the tensor decompositions we discover. In this section we prove such uniqueness claims for the chisel

$$(22) \quad \mathcal{C}_{\text{uni}} = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}.$$

The proof uses the fact that $\text{Der}(\mathcal{C}_{\text{uni}}, \Gamma)$ has the structure of a Lie algebra. Similar uniqueness results hold for other chisels $\mathcal{C}$ for which $\text{Der}(\mathcal{C}, \Gamma)$ has a nice algebraic structure—all of the chisels introduced in Section 7 are of this type.

9.1. The universal chisel. We begin though by explaining why we consider $\mathcal{C}_{\text{uni}}$ to be the default chisel and refer to it as ‘universal’. The following result, a special case of [16, Theorem 1], asserts that $\mathcal{C}_{\text{uni}}$ is the least restrictive chisel: if Γ has only scalar $\mathcal{C}_{\text{uni}}$ -derivations, then it has only scalar $\mathcal{C}$ -derivations for *any* chisel $\mathcal{C}$. To coin a phrase, there is one chisel to rule them all.

Proposition 9.1. *Let Γ be a tensor defined over an infinite field $\mathbb{K}$. For any chisel $\mathcal{C}$ and any $\mathcal{C}$ -derivation $(D_1, \dots, D_\ell)$ of Γ , there are nonzero scalars $(\lambda_1, \dots, \lambda_\ell)$ such that $(\lambda_1 D_1, \dots, \lambda_\ell D_\ell)$ is a $\mathcal{C}_{\text{uni}}$ -derivation of Γ .*

Proof. Recall from Remark 5.3 the convention that $D_b = 0$ whenever column b of $\mathcal{C}$ is zero. Hence, we focus attention on the subset $A \subseteq [\ell]$ upon which $\mathcal{C}$ is engaged. For $v \in \mathbb{K}^m$ and $a \in A$ define $\tau_{av} := (v\mathcal{C})_a = \sum_j v(j)\mathcal{C}_{ja}$. For each $a \in [\ell]$, there exists j with $\mathcal{C}_{ja} \neq 0$. Hence, $0 = \tau_{av} = \sum_i v(j)\mathcal{C}_{ja}$ is a nonzero homogeneous system of linear equations and its solution space is a hyperplane. Observe that $\tau_{av} \neq 0$ for some $a \in [\ell]$. (Otherwise, $\mathbb{K}^m$ would be a union of finitely many hyperplanes, which is impossible since $\mathbb{K}$ is infinite.) Hence, there exists a $v \in \mathbb{K}^m$ such that $(\tau_{av} : a \in [\ell])$ has no zero entry. Since D is a $\mathcal{C}$ -derivation and therefore also a $v\mathcal{C}$ -derivation, it follows that $(\tau_{av}^{-1} D_a : a \in [\ell])$ is a $\mathcal{C}_{\text{uni}}$ -derivation. $\square$

9.2. Uniqueness. The following result provides the theoretical foundation for our uniqueness claims for the chisel $\mathcal{C}_{\text{uni}}$.

Theorem 9.2. *For a tensor Γ defined over $\mathbb{C}$, the following hold:*

- (a) *Let $(D_a : a \in [\ell])$ be a $\mathcal{C}_{\text{uni}}$ -derivation of Γ . For each $a \in [\ell]$, the operator D_a can be decomposed as $D_a = S_a + N_a$, where S_a is diagonalizable and N_a is nilpotent, and any two such decompositions are conjugate under an invertible operator on U_a . Furthermore, both $(S_a : a \in [\ell])$ and $(N_a : a \in [\ell])$ are $\mathcal{C}_{\text{uni}}$ -derivations of Γ .*
- (b) *Let $(D_a : a \in [\ell])$ and $(E_a : a \in [\ell])$ be $\mathcal{C}_{\text{uni}}$ -derivations of Γ such that all D_a and E_a are diagonalizable. For each $a \in [\ell]$, there are invertible matrices X_a such that D_a and $X_a^{-1} E_a X_a$ are simultaneously diagonalizable.*

Proof. Given $\mathcal{C}_{\text{uni}}$ -derivations $(D_a : a \in [\ell])$ and $(E_a : a \in [\ell])$ and $\lambda \in \mathbb{C}$,

$$(D_a + \lambda E_a : a \in [\ell]) \quad \text{and} \quad (D_a E_a - E_a D_a : a \in [\ell])$$

are both $\mathcal{C}_{\text{uni}}$ -derivations. Thus, the set $\text{Der}(\mathcal{C}_{\text{uni}}, \Gamma)$ of all $\mathcal{C}$ -derivations of Γ is a $\mathbb{C}$ -Lie algebra, and (a) follows from standard theory [17, Chapter III, Theorem 16].

Each $\mathbb{C}$ -Lie algebra has a maximal (*Cartan*) subalgebra of simultaneously diagonalizable elements. Every diagonalizable element lies in a Cartan subalgebra, and two Cartan subalgebras are conjugate [17, Chapter IX, Theorem 3]. In (b), for each $a \in [\ell]$, let H_a be a Cartan subalgebra containing D_a , and let J_a be a

Cartan subalgebra containing E_a . Let X_a be an invertible matrix conjugating J_a to H_a . Then D_a and $X_a^{-1}E_aX_a$ lie in a common Cartan subalgebra, so they are simultaneously diagonalizable. $\square$

Theorem 9.2 identifies the optimal sparsity pattern determined by $\mathcal{C}_{\text{uni}}$. In particular, a single random derivation usually provides all of the information we need:

Corollary 9.3. *Any two $\mathcal{C}_{\text{uni}}$ -derivations of a tensor Γ define a unique joint sparsity pattern.*

Proof. Given two $\mathcal{C}_{\text{uni}}$ -derivations, select their semisimple components from Theorem 9.2(a). Next, use Theorem 9.2(b) to conjugate those semisimple parts in a common Cartan subalgebra. They now have common eigenspaces, and hence define a common sparsity pattern. $\square$

Over $\mathbb{R}$ the situation is the same except that we occasionally need 2×2 blocks with complex eigenvalues, so the resulting sparsity patterns may be larger.

9.3. Ordering the blocks. If $\{X_i : i \in [k]\}$ is the set of eigenspaces of a complex linear operator, the corresponding eigenvalues are returned with an arbitrary ordering. This means that every sparsity pattern we produce is only unique up to reordering. One often has to contend with such issues when seeking sparsity patterns, but our algorithms can produce so many blocks that it behooves us to exert some control when we can.

Proposition 9.4. *If the space of $\mathcal{C}$ -derivations of a tensor has dimension one more than that of the sparsity space of $\mathcal{C}$, and if there is a non-scalar $\mathcal{C}$ -derivation D with all real eigenvalues, then there is a well-defined unique canonical ordering of the associated sparsity pattern.*

Proof. If $D = (D_a : a \in [\ell])$ is such a $\mathcal{C}$ -derivation, then each D_a can be taken to be a diagonal matrix with ordered eigenvalues $\delta_a(1), \dots, \delta_a(k_a)$. As the space of $\mathcal{C}$ -derivations is one-dimensional over the scalar derivations, it follows that any non-scalar derivation E_a has eigenvalues $\{\delta_a(1) + \beta_a, \dots, \delta_a(k_a) + \beta_a\}$ for some scalar β_a . In particular, the ordering of the eigenspaces inherited from eigenvalues is invariant under every choice of non-scalar derivation. $\square$

The conditions of Proposition 9.4 are usually met for a sufficiently generic tensor possessing a nontrivial sparsity pattern $\Delta(\mathcal{C}, \delta)$. Even if they are not, however, one can appeal to more subtle ordering tools from Lie theory. Avoiding a technical digression, one idea is to order the eigenspaces along a composition series and then appeal to highest weight vectors. This approach would handle all but the abelian Lie algebras whose dimension over the scalars is greater than 1; no canonical order is known for such Lie algebras.

10. SUMMARY

We have introduced the concept of *sparsity patterns*, which capture the outputs of many of the tensor clustering algorithms in current use. We also introduced *chisels* to parameterize classes of computable sparsity patterns, and showed that these classes include new patterns not known to be detectable by existing methods. We proved that, over infinite fields, one particular chisel is universal among all chisels. We also described an efficient algorithm to detect the existence of sparsity patterns in tensors, and proved that the resulting patterns are unique.

Acknowledgments. This research began as a challenge from colleagues in applied mathematics to adapt algebraic algorithms designed for fields with exact arithmetic to produce practicable tools for computing with tensors over fields with approximate operations. While these colleagues remain unnamed, we thank them both for their critiques and for their encouragement. We also thank the Departments of Mathematics at Bucknell & Cornell University for jointly hosting our team meetings that led to the discovery of the original chisel method. We thank Nick Vannieuwenhoven for engaging discussions on optimizing chisel algorithms in September 2024. Finally, we thank the anonymous referees for their careful reading and helpful suggestions that led to significant improvements in the paper.

The work was initially supported by the Simons Foundation. P.A. Brooksbank was supported by National Science Foundation Grant DMS 2319372. M.D. Kassabov was supported by National Science Foundation Grant DMS 2319371. J.B. Wilson was supported by National Science Foundation Grant DMS 2319370.

REFERENCES

- [1] Evrim Acar, Seyit A. Çamtepe, M.S. Krishnamoorthy, and Bülent Yener, *Modeling and Multiway Analysis of Chatroom Tensors*, Intelligence and Security Informatics. ISI 2005. Lecture Notes in Computer Science, 2005.
- [2] Evrim Acar, Seyit A. Çamtepe, and Bülent Yener, *Collective Sampling and Analysis of High Order Tensors for Chatroom Communications*, Proceedings of the 4th IEEE International Conference on Intelligence and Security Informatics, 2006, pp. 213–224.
- [3] Jeff and Edelman Bezanson Alan and Karpinski, *Julia: A fresh approach to numerical computing*, SIAM review **59** (2017), no. 1, 65–98.
- [4] Peter A. Brooksbank and Clara R. Chaplin, *New linear invariants of hypergraphs* (2025). arxiv:2512.03342.
- [5] Peter A. Brooksbank and James B. Wilson, *Groups acting on tensor products*, J. Pure Appl. Algebra **218** (2014), no. 3, 405–416, DOI 10.1016/j.jpaa.2013.06.011. MR3124207
- [6] Peter A. Brooksbank, Joshua Maglione, and James B. Wilson, *A package for isomorphism testing of groups and tensors of tame genus*, 2018.
- [7] ———, *Exact sequences of inner automorphisms of tensors*, J. Algebra **545** (2020), 43–63, DOI 10.1016/j.jalgebra.2019.07.006. MR4044688
- [8] P. A. Brooksbank, M. D. Kassabov, and J. B. Wilson, *OpenDleto: Algorithms for structure recovery in tensors*, <https://github.com/TheTensor-space/OpenDleTo>.
- [9] Yunfeng Cai and Chengyu Liu, *An algebraic approach to nonorthogonal general joint block diagonalization*, SIAM J. Matrix Anal. Appl. **38** (2017), no. 1, 50–71, DOI 10.1137/16M1080756. MR3592077
- [10] Yunfeng Cai and Ren-Cang Li, *Perturbation analysis for matrix joint block diagonalization*, Linear Algebra Appl. **581** (2019), 163–197, DOI 10.1016/j.laa.2019.07.012. MR3983068
- [11] Yunfeng Cai, Decai Shi, and Shufang Xu, *A matrix polynomial spectral approach for general joint block diagonalization*, SIAM J. Matrix Anal. Appl. **36** (2015), no. 2, 839–863, DOI 10.1137/130935264. MR3357633
- [12] J.-F. Cardoso, *Super-symmetric decomposition of the fourth-order cumulant tensor. Blind identification of more sources than sensors* **5** (1991), 3109 – 3112 vol.5, DOI 10.1109/ICASSP.1991.150113.
- [13] Lieven De Lathauwer, *Decompositions of a Higher-Order Tensor in Block Terms—Part II: Definitions and Uniqueness*, SIAM Journal on Matrix Analysis and Applications **30** (2008), no. 3, 1033–1066, DOI 10.1137/070690729.
- [14] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle, *A Multilinear Singular Value Decomposition*, SIAM Journal on Matrix Analysis and Applications **21** (2000), no. 4, 1253–1278, DOI 10.1137/S0895479896305696.
- [15] W. Eberly and M. Giesbrecht, *Efficient decomposition of associative algebras over finite fields*, J. Symbolic Comput. **29** (2000), no. 3, 441–458, DOI 10.1006/jsco.1999.0308. MR1751390
- [16] Uriya First, Joshua Maglione, and James B. Wilson, *A spectral theory for transverse tensor operators* (2020). arXiv:1911.02518.

- [17] Nathan Jacobson, *Lie algebras*, Dover Publications, Inc., New York, 1979. Republication of the 1962 original. MR0559927
- [18] Ruhui Jin, Joe Kileel, Tamara G. Kolda, and Rachel Ward, *Scalable symmetric tucker tensor decomposition*, SIAM J. Matrix Anal. Appl. **45** (2024), no. 4, 1746–1781, DOI 10.1137/23M1582928. MR4804192
- [19] Joachim Jelisiejew, J. M. Landsberg, and Arpan Pal, *Concise tensors of minimal border rank*, Math. Ann. **388** (2024), no. 3, 2473–2517, DOI 10.1007/s00208-023-02569-y. MR4705743
- [20] Tamara G. Kolda and Brett W. Bader, *Tensor decompositions and applications*, SIAM Rev. **51** (2009), no. 3, 455–500, DOI 10.1137/07070111X. MR2535056
- [21] Lek-Heng Lim, *Tensors in computations*, Acta Numer. **30** (2021), 555–764, DOI 10.1017/S0962492921000076. MR4298222
- [22] T. Maehara and K. Murota, *A numerical algorithm for block-diagonal decomposition of matrix *-algebras with general irreducible components.*, Japan J. Indust. Appl. Math. **27** (2010), 263–293, DOI 10.1007/s13160-010-0007-8.
- [23] L.R. Tucker, *Some mathematical notes on three-mode factor analysis.*, Psychometrika **31** (1966), 279–311, DOI 10.1007/BF02289464.
- [24] Nick Vannieuwenhoven, *A chiseling algorithm for low-rank Grassmann decomposition of skew-symmetric tensors* (2024), available at 2410.14486.
- [25] James B. Wilson, *Existence, algorithms, and asymptotics of direct product decompositions, I*, Groups Complex. Cryptol. **4** (2012), no. 1, 33–72. MR2921155
- [26] ———, *Decomposing p -groups via Jordan algebras*, J. Algebra **322** (2009), no. 8, 2642–2679. MR2559855
- [27] ———, *Finding central decompositions of p -groups*, J. Group Theory **12** (2009), no. 6, 813–830. MR2582050

DEPARTMENT OF MATHEMATICS, BUCKNELL UNIVERSITY, LEWISBURG, PA 17837, USA
 Email address: pbrooks@bucknell.edu

DEPARTMENT OF MATHEMATICS, 310 MALOTT HALL, ITHACA, NY 14853
 Email address: kassabov@math.cornell.edu

DEPARTMENT OF MATHEMATICS, COLORADO STATE UNIVERSITY, FORT COLLINS, COLORADO, 80523-1874, USA
 Email address: James.Wilson@ColoState.Edu